

Metody numeryczne

Wykład nr 11

Generatory liczb pseudolosowych

Aneta Wróblewska

UMCS, Lublin

Losowość, liczby losowe i pseudolosowe oraz ich wykorzystanie

- W naturze, technice, ekonomii i życiu społecznym często spotykamy zjawiska, które wydają się być losowe.
- Trudność w przewidywaniu ich przyszłych zachowań lub określeniu przyczyn wynika z braku informacji, błędów w obserwacji, czy ograniczeń technicznych w dostępie do danych.
- Przyczyny losowości mogą być związane z właściwościami fizycznymi zjawisk lub ich złożonością, uniemożliwiającą modelowanie deterministyczne.

Losowość w matematyce i kryptografii

- Rozkład liczb pierwszych wśród liczb naturalnych. Chociaż można określić średnią częstość ich występowania, dokładne rozmieszczenie liczb pierwszych jest nieznane i wydaje się losowe. Pojawienie się liczby pierwszej w ciągu liczb naturalnych jest trudne do przewidzenia bez szczegółowej analizy.
- Liczby losowe znajdują zastosowanie w statystycznych badaniach reprezentatywnych, kontroli jakości, badaniach rynkowych oraz w naukach eksperymentalnych do planowania eksperymentów.
- Liczby losowe są niezbędne w metodach Monte Carlo, używanych do obliczeń prawdopodobieństw i optymalizacji.
- W kryptografii, liczby losowe pełnią kluczową rolę jako klucze w szyfrach, zwiększając bezpieczeństwo przesyłanych informacji.

Losowość w symulacjach i systemach komunikacyjnych

- Symulacje komputerowe używają liczb losowych do imitacji realnych procesów opisanych równaniami, uwzględniając losowe czynniki wpływające na te zjawiska.
- Badania te często stanowią jedyną metodę ilościowej analizy złożonych procesów, które nie mogłyby być inaczej zbadane.
- Liczby losowe tworzą złudzenie realizmu w grach komputerowych, symulatorach treningowych oraz w grach strategicznych.
- Rozwój technologii komunikacyjnych, jak telefonia komórkowa i sieci komputerowe, intensyfikuje potrzebę skutecznych generatorów liczb losowych.

- **Ciąg** liczbowy nazywamy **losowym**, jeśli nie istnieje krótszy algorytm, który by go opisał niż sam ciąg.
- Z ciągu takiego nie można wywnioskować żadnych reguł umożliwiających jego odtworzenie bez znajomości wszystkich jego elementów.
- Nie jest też możliwe przewidzenie żadnego elementu ciągu na podstawie pozostałych.

- Mimo że istnieją reguły opisujące ciąg, nasza niewiedza może uniemożliwić ich identyfikację.
- Taki **ciąg** nazywamy **pseudolosowym** i w wielu przypadkach może być traktowany jak ciąg losowy.
- W sytuacjach praktycznych pseudolosowe ciągi liczb mogą efektywnie symulować losowe zachowania.
- W przeciwieństwie do prawdziwie losowych, liczby pseudolosowe generowane są przez algorytmy deterministyczne.
- Ważne jest, aby generowane ciągi liczb były nieprzewidywalne i miały właściwości statystyczne podobne do ciągów losowych.

Przykłady generatorów ciągów liczb pseudolosowych

- **LCG (Linear Congruential Generator)**: Jedna z najstarszych i najprostszych metod generowania liczb pseudolosowych.
- **Mersenne Twister**: Znany z bardzo długiego okresu i wysokiej jakości generowanych ciągów.
- **Cryptographically Secure Pseudorandom Number Generators (CSPRNG)**: Generatory o wysokiej nieprzewidywalności, stosowane w kryptografii.

Historyczne metody otrzymywania liczb losowych

Wprowadzenie do historycznych metod liczb losowych

- Od dawna istnieje zapotrzebowanie na liczby losowe w badaniach statystycznych.
- Jednymi z pierwszych źródeł liczb losowych były tablice liczb losowych.
- Przykłady historycznych zbiorów liczb losowych obejmują różne techniki ich generacji.

- W 1927 roku L.H. Tippett opublikował pierwszą tablicę losowych cyfr, składającą się z 41600 cyfr pochodzących z danych spisu powszechnego w Wielkiej Brytanii.
- W 1939 roku R.A. Fisher i F. Yates wydali tablicę 15000 losowych cyfr, które zaczerpnęli z cyfr od 15 do 19 z tablic logarytmicznych.
- Kendall, Babington i Smith w tym samym roku zaprezentowali tablicę 100000 cyfr losowych uzyskanych za pomocą "elektrycznej ruletki".

Tablice liczb losowych w Polsce i RAND Corporation

- W 1951 roku w Polsce GUS opracował własną tablicę liczb losowych, korzystając z pasków drukujących maszyn liczbowych.
- W 1955 roku RAND Corporation stworzyła tablicę miliona cyfr losowych, używając impulsów binarnych, co umożliwiało łatwe zastosowanie w obliczeniach komputerowych.
- Wady tablic liczb losowych obejmowały ograniczoną długość i konieczność rozwoju algorytmów do generowania nowych ciągów losowych.

Generowanie ciągów liczb losowych na podstawie tablicy cyfr losowych

- 1 Wybór losowej pięciocyfrowej liczby z tablicy.
- 2 Modyfikacja pierwszej cyfry liczby modulo 2. Tak zmieniona liczba pięciocyfrowa wskazuje numer wiersza w tablicy.
- 3 Zredykowana dwucyfrowa końcówka liczby modulo 50 wskazuje numer kolumny.
- 4 Proces rozpoczyna się od wybranej pozycji w tablicy, co tworzy losowy ciąg.

Współczesne metody otrzymywania liczb losowych

- Współczesne metody generacji liczb losowych dzielą się na algorytmiczne i fizyczne.
- Algorytmy matematyczne pozwalają na wielokrotne otrzymanie tego samego ciągu pseudolosowego.
- Generatory fizyczne opierają się na mierzalnych parametrach procesów fizycznych, które zachodzą w sposób losowy.

Generatory fizyczne

Przykłady generatorów fizycznych

- Mechaniczne urządzenia losujące, takie jak moneta, kostka do gry, czy ruletka.
- Licznik Geigera, mierzący promieniowanie jądrowe, które zachodzi losowo.
- Elektroniczne liczniki impulsów, np. z dysków komputerowych, monitorów czy kart dźwiękowych.
- Urządzenia generujące losowe bity z klawiatury lub arytmometru w intensywnie używanych komputerach.
- Specjalnie skonstruowane elektroniczne urządzenia generujące liczby losowe, np. z diod szumowych, często dostępne jako karty komputerowe.

Ważność testowania generatorów fizycznych

- Każdy wygenerowany ciąg liczb losowych musi być testowany przed użyciem, aby zapewnić, że zachowuje on właściwości losowości.
- W przypadku awarii urządzenia, wygenerowane ciągi mogą stracić swoje losowe właściwości.
- Dla celów kryptograficznych, ciągi liczb losowych powinny być zapisywane w dwóch kopiach na zewnętrznych nośnikach danych.

Generatory algorytmiczne

Generatory algorytmiczne

Generowanie liczb z rozkładu jednostajnego

Podstawą algorytmicznego generowania liczb pseudolosowych jest otrzymanie ciągu liczb całkowitych

$$X_i \in \{0, 1, \dots, M - 1\}, \quad i = 1, 2, \dots,$$

które możliwie dobrze imitują losowanie z rozkładu jednostajnego.

Następnie liczby całkowite przekształca się do przedziału $[0, 1)$ według wzoru

$$R_i = \frac{X_i}{M}.$$

Otrzymany ciąg

$$R_i \in [0, 1)$$

traktuje się jako dyskretne przybliżenie rozkładu jednostajnego na przedziale $[0, 1)$.

Przeskalowanie wartości generatora

Przejście do zadanego przedziału

Założmy, że generator zwraca liczby całkowite $X \in \{0, 1, \dots, MAX\}$.

Wartość z przedziału $[0, 1)$ można otrzymać przez: $R = \frac{X}{MAX+1}$.

Wartość całkowitą z przedziału $\{0, 1, \dots, max\}$, gdzie $max < MAX$, można uzyskać ze wzoru: $Y = \left\lfloor \frac{X}{MAX+1} (max + 1) \right\rfloor$.

Wartość całkowitą z przedziału $\{min, min + 1, \dots, max\}$ można uzyskać jako: $Y = min + \left\lfloor \frac{X}{MAX+1} (max - min + 1) \right\rfloor$.

Uwaga praktyczna: operator modulo

W programach często spotyka się prosty zapis:

$$Y = X \bmod (max + 1),$$

który ma zwracać liczby z przedziału $\{0, 1, \dots, max\}$.

Problem

Jeżeli liczba możliwych wartości generatora nie jest podzielna przez $max + 1$, niektóre wyniki mogą pojawiać się częściej niż inne.

Dlatego do prostych ćwiczeń taki zapis bywa akceptowalny, ale w zastosowaniach wymagających dobrej jakości losowości lepiej stosować metody unikające obciążenia modulo.

Generatory kongruencyjne

Najbardziej znanym sposobem generowania liczb pseudolosowych jest metoda opracowana przez Lehmera w 1951 zwana **liniowym generatorem kongruencyjnym** (ang. LCG - Linear Congruential Generator). Można wyróżnić dwa podstawowe wzory do obliczania liczb pseudolosowych z wykorzystaniem generatora LCG.

Addytywny LCG:

$$X_{n+1} = a \cdot X_n + c \mod M$$

Multiplikatywny LCG:

$$X_{n+1} = a \cdot X_n \mod M$$

gdzie:

- X_n - n -ta liczba pseudolosowa
- a - mnożnik
- c - parametr, dla generatora multiplikatywnego $c = 0$
- M - moduł generatora

Generatory kongruencyjne

Właściwości i okres generatora

Dla generatora kongruencyjnego $X_{n+1} = (aX_n + c) \bmod M$ **okres** oznacza liczbę kolejnych wartości, po których ciąg zaczyna się powtarzać.

Maksymalny możliwy okres nie może być większy niż liczba różnych stanów, czyli M .

Generator mieszany, $c \neq 0$

Generator może mieć pełny okres równy M , jeżeli parametry a , c i M spełniają odpowiednie warunki, m.in.:

$$\text{nwd}(c, M) = 1.$$

Dla $M = 2^m$ jednym z typowych warunków jest

$$a \equiv 1 \pmod{4}.$$

Generatory kongruencyjne

Właściwości i okres generatora - cd

Generator multiplikatywny, $c = 0$

Jeżeli $M = p$, gdzie p jest liczbą pierwszą, to dla niezerowego ziarna maksymalny okres wynosi

$$p - 1.$$

Osiąga się go wtedy, gdy a jest pierwiastkiem pierwotnym modulo p .

- Liczby generowane przez addytywny LCG (Linear Congruential Generator) mogą przyjmować wartości z przedziału od 0 do $M - 1$.
- Multiplikatywny LCG generuje wartości z przedziału od 1 do $M - 1$.
- Po M wykonaniach, wartości zaczynają się powtarzać, co wynika z własności działania modulo.

Konfiguracja generatora liczb losowych

- Konieczne jest podanie wartości oznaczającej ziarno (ang. *seed*), które jest początkową wartością X_0 .
- Aby generator działał poprawnie, wartości a , c i M nie mogą być przypadkowe.

Dobór parametrów LCG

Warunki pełnego okresu

Dla generatora mieszanego

$$X_{n+1} = (aX_n + c) \bmod m$$

pełny okres równy m można uzyskać, gdy spełnione są warunki Hulla–Dobella:

- 1 c i m są względnie pierwsze:

$$\text{nwd}(c, m) = 1.$$

- 2 $a - 1$ jest podzielne przez każdy czynnik pierwszy liczby m .
- 3 Jeżeli m jest podzielne przez 4, to $a - 1$ również jest podzielne przez 4.

Spełnienie tych warunków gwarantuje maksymalną długość cyklu, ale nie oznacza jeszcze, że generator ma bardzo dobre własności statystyczne.

Przykład doboru parametrów LCG

Rozważmy generator:

$$X_{n+1} = (aX_n + c) \bmod m$$

oraz parametry:

$$m = 16, \quad a = 5, \quad c = 3.$$

Sprawdzamy warunki pełnego okresu:

- ❶ $\text{nwd}(c, m) = \text{nwd}(3, 16) = 1.$
- ❷ Czynnikiem pierwszym liczby 16 jest 2, a

$$a - 1 = 4$$

jest podzielne przez 2.

- ❸ Ponieważ 16 jest podzielne przez 4, sprawdzamy, czy

$$a - 1 = 4$$

jest podzielne przez 4. Jest.

Generator może więc osiągnąć pełny okres równy $m = 16$.

Jak sprawdzić okres generatora?

Aby sprawdzić okres generatora LCG, można zapamiętywać wygenerowane wartości aż do momentu powtórzenia stanu początkowego.

Idea algorytmu

- 1 Ustal ziarno X_0 .
- 2 Generuj kolejne wartości: $X_{n+1} = (aX_n + c) \bmod m$.
- 3 Zliczaj wygenerowane wartości.
- 4 Zakończ, gdy ponownie pojawi się X_0 .

Jeżeli generator mieszany ma pełny okres, to przed powrotem do X_0 powinien wygenerować m różnych stanów.

Dla małych wartości m można to łatwo sprawdzić ręcznie lub programowo.

D.E.Knuth pokazał, iż niektóre wartości mogą prowadzić do wygenerowania bardzo krótkich cykli, co oczywiście jest wysoce niepożądane. Na przykład dla $b = 19$, $M = 381$ oraz $X_1 = 0$ otrzymamy sekwencję: $0, 1, 20, 0, 1, 20, \dots$

- Główną wadą generatorów liniowych jest ich przewidywalność - punkty w przestrzeni wielowymiarowej mogą układać się na ograniczonej liczbie prostych, płaszczyzn lub hiperpłaszczyzn. To jest klasyczna słabość wykrywana m.in. testem widmowym.

Popularne generatory liczb losowych

Nazwa	M	a	c
Numerical Recipes	2^{32}	1664525	1013904223
Borland C/C++	2^{32}	22695477	1
GNU Compiler Collection	2^{32}	69069	5
ANSI C	2^{32}	1103515245	12345
Borland Delphi, Virtual Pascal	2^{32}	134775813	1
Microsoft Visual/Quick C/C++	2^{32}	214013	2531011
ANSIC	2^{31}	1103515245	12345
MINSTD	$2^{31} - 1$	16807	0

Table: Wybrane generatory liczb losowych

Przykład generatora kongruencyjnego mieszanego

- Parametry generatora:

$$a = 4, \quad c = 2, \quad M = 9, \quad X_0 = 0.$$

- Wartość X_0 jest ziarnem, czyli wartością początkową generatora.
- Kolejne wartości obliczamy ze wzoru:

$$X_n = (4 \cdot X_{n-1} + 2) \bmod 9, \quad n = 1, 2, \dots$$

Rozwiązanie – kolejne wartości X_n

- $X_0 = 0$ – ziarno
- $X_1 = (4 \cdot 0 + 2) \bmod 9 = 2$
- $X_2 = (4 \cdot 2 + 2) \bmod 9 = 1$
- $X_3 = (4 \cdot 1 + 2) \bmod 9 = 6$
- $X_4 = (4 \cdot 6 + 2) \bmod 9 = 8$
- $X_5 = (4 \cdot 8 + 2) \bmod 9 = 7$
- $X_6 = (4 \cdot 7 + 2) \bmod 9 = 3$
- $X_7 = (4 \cdot 3 + 2) \bmod 9 = 5$
- $X_8 = (4 \cdot 5 + 2) \bmod 9 = 4$
- $X_9 = (4 \cdot 4 + 2) \bmod 9 = 0$
- $X_{10} = (4 \cdot 0 + 2) \bmod 9 = 2$
- $X_{11} = (4 \cdot 2 + 2) \bmod 9 = 1$
- $X_{12} = (4 \cdot 1 + 2) \bmod 9 = 6$
- $X_{13} = (4 \cdot 6 + 2) \bmod 9 = 8$
- $X_{14} = (4 \cdot 8 + 2) \bmod 9 = 7$
- $X_{15} = (4 \cdot 7 + 2) \bmod 9 = 3$
- $X_{16} = (4 \cdot 3 + 2) \bmod 9 = 5$
- $X_{17} = (4 \cdot 5 + 2) \bmod 9 = 4$
- $X_{18} = (4 \cdot 4 + 2) \bmod 9 = 0$

Okres generatora wynosi 9, ponieważ po wartości $X_9 = 0$ ciąg zaczyna się powtarzać od tego samego ziarna.

Sam okres nie wystarcza do oceny generatora. Warto sprawdzić także rozmieszczenie kolejnych wartości.

Wizualizacja jakości generatora

Punkty tworzone z kolejnych wartości

Jednym z prostych sposobów oceny generatora jest utworzenie punktów z kolejnych wartości ciągu.

Można wykorzystać pary: (X_0, X_1) , (X_2, X_3) , $(X_4, X_5), \dots$ albo pary sąsiednie: (X_0, X_1) , (X_1, X_2) , $(X_2, X_3), \dots$

Aby narysować punkty w kwadracie jednostkowym, najpierw skalujemy wartości: $R_n = \frac{X_n}{m}$.

Następnie rysujemy punkty: (R_0, R_1) , (R_2, R_3) , $(R_4, R_5), \dots$

Interpretacja

Jeżeli punkty tworzą wyraźne linie, pasy albo regularne wzory, generator może mieć słabe własności statystyczne.

Generator Lehmera, nazywany czasami generatorem liczb losowych Parka-Millera (od Stephena K. Parka i Keitha W. Millera), to rodzaj Liniowego Generatorsa Kongruencyjnego (LCG), który operuje w multiplikatywnej grupie modulo M . Jego działanie określa wzór:

$$X_{k+1} = a \cdot X_k \mod M,$$

gdzie:

- M – liczba pierwsza lub potęga liczby pierwszej,
- a – element mający wysoki rząd modulo M (np. pierwiastek pierwotny),
- X_0 – ziarno, liczba względnie pierwsza z M .

W 1988 roku, Park i Miller zasugerowali wartości $M = 2^{31} - 1$ (liczba Mersenne'a M_{31}) i $a = 7^5 = 16807$ (pierwiastek pierwotny modulo M_{31}), obecnie znane jako MINSTD. Mimo krytyki, te parametry pozostały w użyciu do dzisiaj, włącznie z zastosowaniami w CarbonLib i `minstd_rand0` z C++11. Park, Miller i Stockmeyer zasugerowali później użycie mnożnika $a = 48271$ zamiast 16807.

W Sinclair ZX-81 i jego następach użyto wartości $M = 2^{16} + 1$ (liczba Fermata F_4) i $a = 75$. CRAY użył generatora Lehmera z $M = 2^{48} - 1$ i $a = 44485709377909$. GNU Scientific Library zawiera kilka generatorów Lehmera, w tym MINSTD i RANF.

Generator Lehmera, choć jest specjalnym przypadkiem LCG (z $c = 0$), posiada unikalne ograniczenia i właściwości. W szczególności, ziarno X_0 musi być względnie pierwsze do M . Maksymalny okres generatora Lehmera, kiedy M jest liczbą pierwszą i a jest pierwiastkiem pierwotnym, równa się $M - 1$.

Uogólniony generator liniowy

- Uogólnienie generatora liniowego polega na uwzględnieniu kilku poprzednich wartości przy obliczaniu bieżącej wartości:

$$X_n = a_1X_{n-1} + \dots + a_kX_{n-k} + b \pmod{M}$$

- Przy odpowiednim doborze stałych $a_1, \dots, a_k, b < M$, generator ten może osiągnąć maksymalny okres równy M .
- Uogólniony generator liniowy, mimo większej złożoności, nadal nie nadaje się do zastosowań kryptograficznych.

Generator Fibonacciego

Wprowadzenie

Generator Fibonacciego jest jedną z odmian uogólnionych generatorów liniowych. Opiera się na idei ciągu Fibonacciego, ale obliczenia wykonuje się modulo m :

$$X_n = (X_{n-1} + X_{n-2}) \bmod m, \quad n \geq 2.$$

Do rozpoczęcia działania generatora potrzebne są dwie wartości początkowe:

$$X_0, X_1.$$

Taki generator może mieć lepsze własności niż bardzo proste generatory liniowe, jednak jego wadą są możliwe korelacje między kolejnymi wyrazami ciągu. Oznacza to, że wartości mogą mieć poprawny rozkład, ale nie muszą być wystarczająco niezależne.

Lagged Fibonacci Generator (LFG)

Uogólnienie generatora Fibonacciego

W celu zmniejszenia prostych zależności między kolejnymi wyrazami można uogólnić generator Fibonacciego do postaci zwanej *lagged Fibonacci generator*:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m, \quad n \geq p, \quad p > q \geq 1.$$

Liczby p i q oznaczają opóźnienia generatora. Do rozpoczęcia generowania potrzebne są wartości początkowe:

$$X_0, X_1, \dots, X_{p-1}.$$

Generator można modyfikować, zastępując dodawanie inną operacją, np. odejmowaniem, mnożeniem albo operacją XOR:

$$X_n = (X_{n-p} \diamond X_{n-q}) \bmod m, \quad n \geq p, \quad p > q \geq 1,$$

gdzie $\diamond$ oznacza wybraną operację.

Dla generatora $X_n = (X_{n-p} + X_{n-q}) \bmod m$, ($n \geq p$, $p > q \geq 1$), należy najpierw podać p wartości początkowych: $X_0, X_1, \dots, X_{p-1}$.

Kroki implementacji

- 1 Wybierz moduł m oraz opóźnienia p i q .
- 2 Utwórz tablicę z wartościami początkowymi.
- 3 Dla $n = p, p+1, \dots$ obliczaj: $X_n = (X_{n-p} + X_{n-q}) \bmod m$.
- 4 Zapisuj kolejne wartości do tablicy lub wypisuj je na ekran.

Wartości początkowe powinny być dobrane tak, aby ciąg nie był trywialny, np. nie składał się wyłącznie z zer.

Przykład zastosowania generatora

Rozważmy generator:

$$X_n = (X_{n-p} + X_{n-q}) \bmod m,$$

gdzie

$$m = 17, \quad p = 3, \quad q = 1.$$

Wartości początkowe:

$$X_0 = 7, \quad X_1 = 16, \quad X_2 = 5.$$

$$X_3 = (X_0 + X_2) \bmod 17 = (7 + 5) \bmod 17 = 12,$$

$$X_4 = (X_1 + X_3) \bmod 17 = (16 + 12) \bmod 17 = 11,$$

$$X_5 = (X_2 + X_4) \bmod 17 = (5 + 11) \bmod 17 = 16.$$

Kolejne wartości ciągu:

$$7, 16, 5, 12, 11, 16, 11, 5, 4, 15, 3, \dots$$

Generator Fibonacciego można rozbudować, wykorzystując więcej poprzednich wartości ciągu:

$$X_n = (X_{n-p_1} \diamond X_{n-p_2} \diamond \dots \diamond X_{n-p_k}) \bmod m,$$

gdzie

$$n \geq p_k, \quad p_k > p_{k-1} > \dots > p_1 \geq 1.$$

Symbole $p_1, p_2, \dots, p_k$ oznaczają opóźnienia, a $\diamond$ oznacza wybraną operację, np. dodawanie, odejmowanie, mnożenie lub XOR.

Przykładami generatorów tego typu są generatory Marsagli, np. Marsa-LFIB4, oraz generator Ziffa, które wykorzystują różne operacje i różne zestawy opóźnień.

Kwadratowy generator kongruencyjny

Aby uniknąć prostej zależności liniowej między kolejnymi wartościami ciągu, można zastosować zależność kwadratową:

$$X_{n+1} = (aX_n^2 + bX_n + c) \bmod m.$$

Parametry a , b , c , moduł m oraz wartość początkowa X_0 muszą być dobrane tak, aby generator miał możliwie długi okres i dobre własności statystyczne.

Dla odpowiednich wartości parametrów maksymalny okres generatora może być równy m .

Generator wykorzystujący wielomiany permutacyjne

Generator może wykorzystywać wielomian

$$g(x) = \sum_{k=0}^r a_k x^k, \quad a_k \in \{0, 1, \dots, m-1\},$$

który jest wielomianem permutacyjnym modulo m .

Oznacza to, że funkcja $x \mapsto g(x) \bmod m$ przestawia elementy zbioru $\{0, 1, \dots, m-1\}$.

Dzięki temu, przy odpowiednim sposobie generowania kolejnych argumentów, można uzyskać ciąg o okresie równym m .

Uwaga

Generator tego typu może ograniczać proste zależności liniowe występujące w klasycznych generatorach kongruencyjnych, ale nadal jest algorytmem deterministycznym. Nie należy automatycznie traktować go jako generatora kryptograficznie bezpiecznego.

Generator inwersyjny, używany do generowania liczb losowych o równomiernym rozkładzie, wykorzystuje wzór:

$$X_{n+1} = \begin{cases} (aX_n^{-1} + b) \bmod p & \text{dla } X_n \neq 0; \\ b & \text{dla } X_n = 0; \end{cases}$$

gdzie p jest liczbą pierwszą. Maksymalny okres generatora, przy odpowiednich wartościach a i b , może wynosić $p - 1$.

Nowoczesne generatory do symulacji numerycznych

Współczesne generatory stosowane w symulacjach nie są zwykłe prostymi LCG. Celem jest uzyskanie długiego okresu, dobrych własności statystycznych oraz wydajnej implementacji.

Przykładowe rodziny generatorów:

- **Mersenne Twister** – bardzo długi okres, popularny w wielu bibliotekach, ale duży stan i brak bezpieczeństwa kryptograficznego.
- **PCG** (*Permuted Congruential Generator*) – połączenie prostego mechanizmu kongruencyjnego z permutacją bitów wyjściowych.
- **xoshiro/xoroshiro** – bardzo szybkie generatory oparte na operacjach XOR, przesunięciach i rotacjach bitowych.

W praktyce należy odróżniać generator do symulacji od generatora do kryptografii.

Generator PCG – idea

Generator PCG łączy dwie idee:

- 1 prostą rekurencję kongruencyjną dla stanu wewnętrznego,
- 2 dodatkową permutację bitów przed zwróceniem wyniku.

Schemat działania:

$$X_{n+1} = (aX_n + c) \bmod 2^m,$$

a następnie:

$$R_n = \text{permute}(X_n).$$

Dlaczego to pomaga?

Sama rekurencja kongruencyjna może mieć widoczne zależności liniowe. Permutacja bitów wyjściowych utrudnia pojawianie się prostych wzorców w obserwowanym ciągu.

PCG jest dobrym przykładem generatora do symulacji, ale nie jest standardowym generatorem kryptograficznym.

Generatory kryptograficznie bezpieczne

W zastosowaniach kryptograficznych nie wystarcza dobry rozkład statystyczny. Generator powinien być odporny na przewidywanie kolejnych bitów nawet wtedy, gdy przeciwnik zna część wcześniejszych wyników.

W praktyce stosuje się generatory typu **CSPRNG** lub **DRBG** (*Deterministic Random Bit Generator*), np.:

- **Hash_DRBG** – oparty na funkcjach skrótu,
- **HMAC_DRBG** – oparty na konstrukcji HMAC,
- **CTR_DRBG** – oparty na szyfrze blokowym w trybie licznikowym.

Ważna zasada praktyczna

Do symulacji numerycznych można używać szybkich generatorów statystycznych. Do kluczy, soli, tokenów i haseł należy używać generatorów kryptograficznych dostarczanych przez system operacyjny lub sprawdzone biblioteki kryptograficzne.

Metody testowania generatorów

Aby ocenić, czy przyjąć, czy odrzucić generator lub wygenerowany ciąg bitów stosujemy w praktyce zestawy odpowiednich testów statystycznych. Testy te powinny potwierdzać:

- równomierny rozkład ciągu bitów,
- losowość rozkładu,
- niezależność kolejnych bitów.

Metody testowania generatorów

Literatura i Testy

W literaturze dostępnych jest wiele testów do oceny generatorów liczb losowych. Można wyróżnić testy ogólne, dotyczące rozkładów liczb całkowitych, oraz specyficzne, skoncentrowane na ciągach binarnych. Zestawy testów powinny być kompletne, ale także wystarczająco ograniczone, aby były wykonalne w praktyce podczas użytkowania generatora.

Historyczne testy FIPS dla ciągów bitowych

Praktyczne zastosowanie testów

W praktycznej weryfikacji generatorów liczb pseudolosowych stosowano m.in. proste testy statystyczne opisane w starszych wymaganiach FIPS dla próbek długości 20 000 bitów.

Przykładowe testy:

- 1 **Test monobitowy** – sprawdza, czy liczba jedynek w ciągu mieści się w określonych granicach.
- 2 **Test pokerowy** – bada równomierność rozkładu bloków czterobitowych.
- 3 **Test serii** – analizuje długości kolejnych serii takich samych bitów.
- 4 **Test długich serii** – sprawdza, czy w ciągu nie pojawiają się zbyt długie serie jednego bitu.

Obecnie w zastosowaniach kryptograficznych większe znaczenie mają standardy dotyczące konstrukcji generatorów, źródeł entropii oraz ciągłego monitorowania ich działania.

Test monobitowy jest podstawowym testem statystycznym używanym do sprawdzania generatorów liczb losowych. Jego głównym celem jest ocena, czy liczba jedynkowych bitów ('1') w wygenerowanym ciągu bitów leży w granicach, które są statystycznie prawdopodobne dla równomiernie losowego ciągu. Test wykorzystuje statystykę chi-kwadrat do oceny, czy zaobserwowane odchylenie liczby jedynek od oczekiwanej średniej jest statystycznie istotne.

Procedura testu monobitowego składa się z kilku kroków:

- 1 Obliczenie liczby jedynek w ciągu 20000 bitów:

$$X = \text{liczba jedynek w ciągu 20000 bitów}$$

- 2 Ocena, czy wygenerowany ciąg można uznać za akceptowalny, na podstawie kryterium:

$$9725 < X < 10275$$

Jeżeli liczba jedynekowych bitów spełnia ten warunek, ciąg jest uznawany za zgodny z oczekiwaniami dla równomiernego rozkładu.

Test pokerowy to metoda statystyczna używana do sprawdzania generatorów liczb losowych, zwłaszcza w kontekście ich zdolności do produkcji ciągów bitów, które nie wykazują widocznych wzorców i są równomiernie rozłożone. Test skupia się na analizie segmentów czterobitowych w ciągu bitów. Jest to krytyczne dla zastosowań, które wymagają wysokiej jakości losowości, takich jak algorytmy kryptograficzne.

Procedura testu pokerowego obejmuje następujące kroki:

- 1 Dzielenie ciągu 20000 bitów na 5000 segmentów czterobitowych.
- 2 Liczenie częstości pojawienia się każdego z 16 możliwych czterobitowych segmentów (od 0000 do 1111).
- 3 Obliczanie statystyki chi-kwadrat z wyników częstości:

$$X = \frac{16}{5000} \left(\sum_{i=0}^{15} [f(i)]^2 \right) - 5000$$

gdzie $f(i)$ to liczba wystąpień i -tego czterobitowego segmentu.

- 4 Ocena zgodności rozkładu segmentów z oczekiwanym rozkładem równomiernym, gdzie zakres akceptowalnych wartości statystyki X wynosi:

$$2,16 < X < 46,17$$

Test serii ocenia, czy liczba serii różnej długości w ciągu bitów jest zgodna z oczekiwaniami dla ciągu bitów o równomiernym rozkładzie losowości. Serią nazywamy ciąg kolejnych bitów o tej samej wartości.

Test serii pozwala zidentyfikować ciągi bitów, które mogą nie wykazywać oczekiwanej losowości ze względu na nieodpowiednią liczbę lub rozkład długości serii. Jest to kluczowe dla zastosowań kryptograficznych i innych, gdzie jakość losowości bezpośrednio wpływa na bezpieczeństwo i efektywność systemów.

Test serii skupia się na analizie długości serii bitów w wygenerowanym ciągu:

- ❶ Podział ciągu 20000 bitów na serie jednorodnych bitów (1 lub 0).
- ❷ Liczenie liczby serii o różnej długości. Specyficzne kategorie długości to 1, 2, 3, 4, 5 oraz 6 i więcej.
- ❸ Porównanie wyników z teoretycznymi wartościami oczekiwanymi dla idealnie losowego ciągu bitów. Akceptowalne zakresy dla liczby serii różnych długości są następujące:
 - Długość serii 1: 2343 do 2657
 - Długość serii 2: 1135 do 1365
 - Długość serii 3: 542 do 708
 - Długość serii 4: 251 do 373
 - Długość serii 5: 111 do 201
 - Długość serii 6 i więcej: 111 do 201

Różne rodzaje testów statystycznych

W literaturze można znaleźć różne grupy testów, które badają rozmaite cechy badanej populacji oraz warunki eksperymentu losowego, w tym:

- Testy losowości, które określają, czy ciąg wyników może być traktowany jako ciąg zmiennych losowych.
- Testy zgodności, które określają, czy obserwacje mają ten sam rozkład prawdopodobieństwa.
- Testy normalności, sprawdzające, czy dane pochodzą z rozkładu normalnego.
- Testy dotyczące parametrów rozkładu, potwierdzające wiarygodność estymowanych parametrów.
- Testy niezależności, które określają, czy zmienne są niezależne.

Podsumowanie - kryteria dobrego generatora

- **Okres:** Idealny generator miałby okres równy lub zbliżony do maksymalnej możliwej liczby unikatowych stanów, co oznacza, że ciąg wartości nie powtarza się przez bardzo długi czas.
- **Równomierność:** Równomierność odnosi się do równego prawdopodobieństwa każdej z możliwych wartości w zakresie generatora. Równomierność generatora można ocenić za pomocą testów statystycznych, takich jak test chi-kwadrat, które pozwalają określić, czy rozkład generowanych liczb jest bliski idealnemu rozkładowi równomiernemu.
- **Nieprzewidywalność:** Nieprzewidywalność oznacza, że nie jest możliwe efektywne przewidzenie kolejnych wartości w ciągu na podstawie jakiejkolwiek skończonej liczby wcześniejszych wartości. Nieprzewidywalność można oceniać przez analizę korelacji i innych wzorców w wygenerowanych ciągach. Testy takie jak test następnego bitu są przykładami metod oceny tego kryterium.